

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a821a14d26d2a613a.jsonl`
- SHA-256: `df87140112c075fee0e0cb44671940d791516b4ba06d9112c2e001ab9fdf4316`
- Source modified: `2026-07-06T17:29:46+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T17:29:03.350Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to REFUTED if you cannot concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if you can trace a concrete input/state that produces the claimed wrong behavior in the code AS WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after reading, mark UNCERTAIN.

FINDING (from the completeness lens):

severity=low | backend/pipeline/segmenters/trajectory_hybrid.py:463
summary: Phase-3 where the AI provider errors on EVERY candidate window yields an empty ai_segments and returns []/status="success", so a run that detected rallies but got zero usable AI boundaries is indistinguishable from a genuinely empty match.
detail: In the AI-handoff loop, a provider error per window is only logged (line 427-433, metrics['error']) and the window contributes []. If every window errors (capacity 503s / rate limits / all clips fail extraction), ai_segments stays empty and process_video falls through Phase 4 to return [] ? status="success". This is a real 'ran, but we don't actually know' case that still masquerades as an empty match, but (a) it is pre-existing, (b) it is shared identically with yolo_hybrid (line 356) and the gemini segmenter (which surface it via loud logs + run_signals counters, not a Failure), and (c) fixing it (e.g. failing the run when all handoff windows errored) is a broader design decision beyond the wf294 detector-timeout scope. Flagging for awareness, not as a blocker for this change.
failure_scenario: A trajectory hybrid detects N>0 candidate windows, but the provider returns metrics['error'] (503/429) for all N handoff clips. ai_segments is empty, process_video returns [], cmd_infer writes status="success"/0-segments; the incomplete run is served as a complete empty result. Lower likelihood/scope than the config-abort and yolo gaps above.
suggested_fix: Out of scope for wf294's minimal fix; if pursued later, track per-window provider errors and return a Failure (or a Success-with-warnings) when every handed-off window errored, mirroring the loud run_signals counters gemini already emits (generate_failed_chunks / oversize_clips_skipped).

Verify against the real code and return your verdict.

2. assistant (2026-07-06T17:29:05.173Z)

I'll verify this finding by examining the actual code.

3. user (2026-07-06T17:29:07.179Z)

```
400             else:
401                 handoff_dir = os.path.join(
```

```

402         output_base(self.config), f"chunks_{provider_name}_handoff"
403     )
404     os.makedirs(handoff_dir, exist_ok=True)
405     logger.info(
406         "Phase 3: %s validation on %d candidate windows...",
407         provider_name,
408         len(handoff_indices),
409     )
410
411     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
412         w_start = max(0, window["start"] - handoff_padding)
413         w_end = min(video_duration, window["end"] + handoff_padding)
414         w_dur = w_end - w_start
415         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
416         if not extract_video_chunk(
417             video_path, w_start, w_dur, w_path, reencode=True
418         ):
419             return []
420         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
421         segments, metrics = provider.analyze_video(
422             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
423         )
424         # B2: a provider failure returns [] with metrics["error"] set ? that is
425         # "we don't know", NOT "no rallies in this window". Surface it loudly so
426         # partial run isn't mistaken for a complete one (mirrors the gemini
427         # segmenter).
428         if metrics.get("error"):
429             logger.warning(
430                 "[Handoff %d] provider error ? window %d skipped: %s",
431                 wi + 1,
432                 wi,
433                 metrics["error"],
434             )
435         valid = []
436         for seg in segments:
437             try:
438                 seg["start_time"] = float(seg["start_time"]) + w_start
439                 seg["end_time"] = float(seg["end_time"]) + w_start
440                 seg["_candidate_id"] = candidate_ids[wi]
441                 valid.append(seg)
442             except (ValueError, TypeError, KeyError) as e:
443                 logger.warning(
444                     "Dropping segment with unparseable timestamps: %s ? %s",
445                     seg,
446                     e,
447                 )
448         try:
449             os.remove(w_path)
450         except OSError:
451             pass
452         return valid
453
454     with concurrent.futures.ThreadPoolExecutor(
455         max_workers=ai_max_workers
456     ) as executor:
457         futures = [
458             executor.submit(
459                 process_candidate_window, wi, all_candidate_windows[wi]
460             )
461             for wi in handoff_indices
462         ]
463         for future in futures:
464             ai_segments.extend(future.result())
465     try:

```

```

465         os.rmdir(handoff_dir)
466     except OSError:
467         pass
468     logger.info(
469         "Phase 3 completed. AI confirmed %d semantic play segments.",
470         len(ai_segments),
471     )
472
473     # --- Phase 4: DB population + candidate linking ---
474     segments_data = []
475     for segment in ai_segments:
476         try:
477             start = float(segment["start_time"])
478             end = float(segment["end_time"])
479             dur = end - start
480             metadata = {
481                 "shot_count": self._shot_count_from(segment, dur),
482                 "shot_count_confidence": segment.get(
483                     "shot_count_confidence", "Unknown"
484                 ),
485                 "detector": f"{self.family}-hybrid-{model_name}",
486             }
487             segment_id = self.db.add_play_segment(
488                 video_id, start, end, None, None, metadata
489             )
490             if segment_id:
491                 candidate_id = segment.get("_candidate_id")
492                 if candidate_id is not None:
493                     self.db.link_candidate_to_segment(candidate_id, segment_id)
494                 segments_data.append(
495                     {
496                         "id": segment_id,
497                         "start_time": start,
498                         "end_time": end,
499                         "duration": dur,
500                         "metadata": metadata,
501                     }
502                 )
503             except Exception as e:
504                 logger.warning("Skipping invalid segment struct: %s", e)
505
506     return segments_data
507

```

4. user (2026-07-06T17:29:08.796Z)

```

1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..f96b495 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +

```

```

19     ## Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Partial adoption of the Result contract (the ABC declares Success|Failure).
34     +     # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35     +     # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
36     +     # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37     +     # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
38     +     # the bare-list success/empty paths are the documented incremental gap
39     +     # --- Sport profile + AI provider wiring (identical across segmenters) ---
40     +     sport_name = self.config.sport
41     @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
42     ok, msg = runner.healthcheck()
43     if not ok:
44         logger.error("%s detector environment not ready: %s", label, msg)
45         return []
46         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
47         # failed/retryable instead of writing an empty-but-"successful" result.
48         return Failure(
49             message=f"{label} detector environment not ready: {msg}",
50             is_recoverable=True,
51         )
52         logger.info("%s detector env OK: %s", label, msg)
53     # --- Phase 2: trajectory -> candidate rally windows ---
54     @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
55     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
56     if not csv_path:
57         logger.error("%s produced no trajectory; aborting.", label)
58         return []
59         # The cold-start / timeout failure mode (e.g. WASB native inference timed
60         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
61         # a legitimately empty match ? so return a Failure the worker marks as
62         # rather than a silent 0-segment "success" (the wf294 incident).
63         return Failure(
64             message=(
65                 f"{label} detector produced no trajectory (timed out or aborted) ?
66                 "
67                 "detector failure, not a 0-rally video."
68             ),
69             is_recoverable=True,
70         )
71     points = runner.parse_trajectory_csv(csv_path)
72     logger.info(

```

```

75     diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76     index 461d7ca..88c50f6 100644
77     --- a/backend/worker/__main__.py
78     +++ b/backend/worker/__main__.py
79     @@ -114,6 +114,42 @@ def _write_report_json(
80         )
81
82
83     +def _serialize_and_push_outputs(
84     +    scratch: str,
85     +    out_uri: str,
86     +    video_id: str,
87     +    segments: List[dict],
88     +    status: str,
89     +    storage_cfg: dict,
90     +    video_uri: str = "",
91     +) -> List[str]:
92     +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93     +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94     +
95     +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96     +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97     +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98     +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99     +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100    +    out_local = os.path.join(scratch, "outputs", video_id)
101    +    os.makedirs(out_local, exist_ok=True)
102    +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103    +    _write_report_json(
104    +        video_id,
105    +        segments,
106    +        status,
107    +        os.path.join(out_local, "report.json"),
108    +        video_uri=video_uri,
109    +    )
110    +    out_prefix = out_uri.rstrip("/")
111    +    pushed: List[str] = []
112    +    for fn in sorted(os.listdir(out_local)):
113    +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114    +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115    +        pushed.append(uri)
116    +    return pushed
117    +
118    +
119    def _run_startup_checks(config: Any) -> bool:
120        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122    @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123        )
124
125        def _fail(rc: int) -> int:
126            - # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127            - # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128            - # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129            + # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the

```

```

130 + # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131 + # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132 + # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133 + # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134 + # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135 + # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136 + try:
137 +     _serialize_and_push_outputs(
138 +         scratch,
139 +         args.out_uri,
140 +         video_id,
141 +         [],
142 +         "failed",
143 +         storage_cfg,
144 +         str(getattr(args, "video_uri", "") or ""),
145 +     )
146 + except Exception as e: # never mask the real failure on an artifact-push
hiccup
147 +     logger.warning("[worker] could not push failure artifacts: %s", e)
148 + # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149 + if getattr(args, "done_uri", None):
150 +     _write_done_marker(
151 +         args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152 @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153     )
154
155     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156 - out_local = os.path.join(scratch, "outputs", video_id)
157 - os.makedirs(out_local, exist_ok=True)
158 - _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159 - _write_report_json(
160 + pushed = _serialize_and_push_outputs(
161 +     scratch,
162 +     args.out_uri,
163 +     video_id,
164 +     segments,
165 +     status,
166 -     os.path.join(out_local, "report.json"),
167 -     video_uri=str(getattr(args, "video_uri", "") or ""),
168 +     storage_cfg,
169 +     str(getattr(args, "video_uri", "") or ""),
170     )
171
172 - out_prefix = args.out_uri.rstrip("/")
173 - pushed = []
174 - for fn in sorted(os.listdir(out_local)):
175 -     uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176 -     storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177 -     pushed.append(uri)
178 -
179     print(
180         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181     )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186 @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB

```

```

population/linking
187     from unittest.mock import MagicMock, patch
188
189     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193     def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197         res = seg.process_video("v1", "clip.mp4")
198         - assert res == []
199         + # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200         + # instead of writing an empty "success".
201         + assert isinstance(res, Failure)
202         + assert "environment not ready" in res.message
203         + # Should bail before ever running inference.
204         + mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210         - assert seg.process_video("v1", "clip.mp4") == []
211         + res = seg.process_video("v1", "clip.mp4")
212         + # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213         + # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214         + assert isinstance(res, Failure)
215         + assert "no trajectory" in res.message and res.is_recoverable
216         +
217         +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220         + mock_runner_cls,
221         + mock_probe,
222         + mock_dur,
223         + mock_extract,
224         + mock_provider_registry,
225         + mock_env,
226         +):
227         + """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228         + parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229         + (0-rally "success"), NOT a Failure."""
230         + mock_env.return_value = "dummy_key"
231         + mock_dur.return_value = 30.0
232         + mock_extract.return_value = True
233         + mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234         + mock_provider_registry.get.return_value = MagicMock()
235         +
236         + seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237         + res = seg.process_video("v1", "clip.mp4")
238         + assert res == [] and not isinstance(res, Failure)
239         + # The detector DID run (a genuine empty, not an abort).
240         + mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243     @_patched
244     diff --git a/tests/test_worker.py b/tests/test_worker.py

```

```

245     index 96edbee..63cc18e 100644
246     --- a/tests/test_worker.py
247     +++ b/tests/test_worker.py
248     @@ -10,6 +10,7 @@ import json
249         import pytest
250
251         from backend.config import load_config
252     +from backend.results import Failure
253         from backend.worker import __main__ as wmain
254
255         # ----- pure helpers
256     @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259         class _EmptySeg:
260             - """A segmenter that finds nothing ? the cold-start failure mode."""
261             + """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263             def __init__(self, db, config):
264                 pass
265     @@ -200,6 +201,20 @@ class _EmptySeg:
266                 return []
267
268
269     +class _FailSeg:
270     +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271     +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272     +
273     +    def __init__(self, db, config):
274     +        pass
275     +
276     +    def process_video(self, video_id, path, max_frames=None):
277     +        return Failure(
278     +            message="WASB detector produced no trajectory (timed out or aborted)",
279     +            is_recoverable=True,
280     +        )
281     +
282     +
283     def test_setup_logging_levels():
284         import logging
285
286     @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector
289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290         - assert (
291         -     json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292         -         "segment_count"
293         + # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294         + # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295         + report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296         + assert report["segment_count"] == 0 and report["status"] == "success"
297         +
298         +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +    tmp_path, monkeypatch
301     +):
302     +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with

```

```

304 + status="failed" so the control plane / alpha-user surface can show an error /
retry. """
305 + monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306 + monkeypatch.setattr(
307 +     wmain.validator_registry,
308 +     "run_all_with_context",
309 +     lambda path, cfg: ([_OKResult()], {}),
310 + )
311 + monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312 + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313 + video = tmp_path / "in.mp4"
314 + video.write_bytes(b"FAKE")
315 + out = tmp_path / "bucket"
316 + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317 +
318 + rc = wmain.main(
319 +     [
320 +         "infer",
321 +         "--video-uri",
322 +         str(video),
323 +         "--out-uri",
324 +         str(out),
325 +         "--scratch",
326 +         str(tmp_path / "s"),
327 +         "--segmenter",
328 +         "wasb_hybrid",
329 +         "--done-uri",
330 +         str(done),
331 +     ]
332 +     == 0
333 + )
334 + # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335 + assert rc == 3
336 + # report.json is written as an explicit FAILURE, not a false empty "success".
337 + report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338 + assert report["status"] == "failed" and report["segment_count"] == 0
339 + assert report["segments"] == []
340 + # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341 + marker = json.loads(done.read_text())
342 + assert marker["returncode"] == 3 and marker["status"] == "failed"
343 + assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347

```

5. assistant (2026-07-06T17:29:11.510Z)

Now let me examine the caller side to understand whether an empty `ai_segments` returns `[]` with `status="success"`, and verify the finding's scope claims.

6. user (2026-07-06T17:29:13.434Z)

```

340         f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
341         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
342         f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
343     )
344
345     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
346     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
347     # persistence and the handoff gate below.

```

```

348         window_stats = [
349             self._enrich_candidate_stats(
350                 cw, points, fps, frame_width, video_path, idx_cfg
351             )
352             for cw in all_candidate_windows
353         ]
354
355     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
356     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
357     if store_candidates:
358         for i, cw in enumerate(all_candidate_windows):
359             candidate_ids[i] = self.db.add_candidate_segment(
360                 video_id,
361                 detector=self.name,
362                 start_time=cw["start"],
363                 end_time=cw["end"],
364                 chunk_index=cw["chunk_index"],
365                 confidence=min(1.0, cw["peak_velocity"]),
366                 stats=window_stats[i],
367                 detection_params=detection_params,
368             )
369
370     if not all_candidate_windows:
371         return []
372
373     # --- Phase 3: AI provider handoff over padded candidate clips ---
374     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
375
376     # candidates above stay the full superset ? only the served play_segments are
377     # gated.
378     handoff_indices = self._select_for_handoff(
379         all_candidate_windows, window_stats, idx_cfg
380     )
381
382     ai_segments: List[dict] = []
383     if skip_ai:
384         # Fully-local: the selected candidate windows ARE the rallies ? emit them
385         # (no clip extraction, no upload). Boundaries are the local detector's; shot
386         # falls back to the duration-based estimate (no AI exchange count).
387         ai_segments = [
388             {
389                 "start_time": all_candidate_windows[wi]["start"],
390                 "end_time": all_candidate_windows[wi]["end"],
391                 "shuttle_exchange_count": None,
392                 "shot_count_confidence": "LocalNoAI",
393                 "_candidate_id": candidate_ids[wi],
394             }
395             for wi in handoff_indices
396         ]
397     else:
398         handoff_dir = os.path.join(
399             output_base(self.config), f"chunks_{provider_name}_handoff"
400         )
401         os.makedirs(handoff_dir, exist_ok=True)
402         logger.info(
403             "Phase 3: %s validation on %d candidate windows...",
404             provider_name,
405             len(ai_segments),
406         )
407

```

```
408         len(handoff_indices),
409     )
```

7. assistant (2026-07-06T17:29:25.000Z)

The finding is confirmed as accurately describing the code. Let me verify the key claims: (1) the per-window error only logs and returns [] (lines 427-433, 419), (2) if all windows error, ai_segments stays empty and returns [] (line 506) as a success. Let me verify the caller writes status="success" for empty segments.

8. user (2026-07-06T17:29:26.437Z)

```
300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "
318             f"(have {list(segmenter_registry.list_available())})"
319         )
320         return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
```

```

346             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347             "detections, or the input resolution differs from the tuned proxy
resolution. "
348             "Re-run with -v for the native command + frame counts.",
349             video_id,
350             segmenter_actual,
351             detector_impl,
352         )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(

```

9. assistant (2026-07-06T17:29:39.168Z)

Confirmed the caller behavior. When `process\_video` returns a bare `[]` (empty ai_segments), `isinstance(result, Failure)` is False, so `status = "success"` (line 330), and it proceeds to write report.json with status="success" and 0 segments.

The finding accurately describes the code as written:

- Line 419: extraction failure returns `[]`.
- Lines 427-433: provider error only logs a warning; the window still contributes `[]` (segments would be empty from the failed provider call).
- Line 463: `ai\_segments.extend(future.result())` ? if all windows error/fail, ai_segments stays empty.
- Line 506: returns `segments\_data` (empty) ? a bare list, not a Failure.
- Worker line 326-330: bare `[]` is treated as success.

This is exactly the "ran but we don't know" case masquerading as an empty match. The finding itself correctly self-assesses this as (a) pre-existing, (b) shared with yolo_hybrid, and (c) out-of-scope for the wf294 detector-timeout fix, and explicitly flags it at severity=low "for awareness, not as a blocker."

The finding is CONFIRMED as an accurate description of a real code behavior. The severity=low and the out-of-scope framing are appropriate ? the diff's wf294 fix deliberately addresses only detector-level failures (healthcheck, run_predict timeout), not per-window AI provider errors, so this gap genuinely remains in the code as written.

10. user (2026-07-06T17:29:46.763Z)

Structured output provided successfully